



EJERCICIO 2 · ALGORITMOS

Percentil **ATP**

Partición de Lomuto y Quickselect aplicados a estadísticas reales del ATP

Juan Pablo Carrera Martinez · Diego Sanchez Lopez

Repositorio: github.com/juanpablocarreram/ProyectoAlgoritmos



¿En qué percentil está un tenista?



Con datos reales del top 50 ATP (edad, puntos de ranking, altura, aces por partido, % de primer saque) queremos ubicar a un jugador o un valor propio dentro de un percentil.



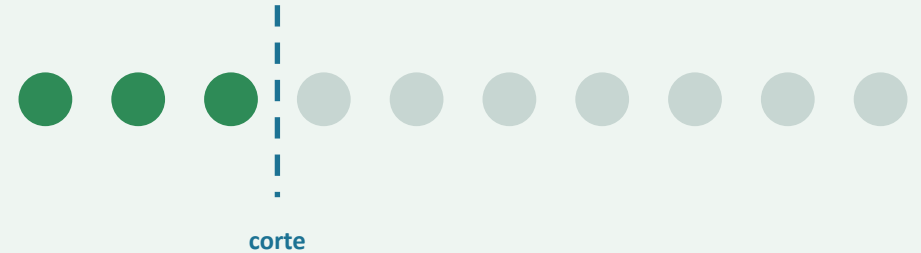
El usuario elige la estadística y mueve un slider de percentil (0–100); la app resalta quién queda dentro y quién queda fuera del corte.



Calcular ese valor de corte es, en el fondo, encontrar el k-ésimo elemento más pequeño de un arreglo — sin necesidad de ordenarlo por completo.



Percentil 70 · Aces por partido



- Dentro del percentil elegido
- Fuera del percentil elegido

Entrada: 50 jugadores · Interfaz web con slider (FastAPI + HTML/CSS/JS)

La partición de Lomuto, en una pasada

Reordena el arreglo alrededor de un pivote: menores o iguales a la izquierda, mayores a la derecha.

Antes



pivote = último elemento

Después



$\leq$ pivote

$>$ pivote

1

Recorre el segmento una sola vez, comparando cada elemento contra el pivote.

2

Cuando encuentra uno $\leq$ pivote, lo intercambia hacia la frontera izquierda.

3

Al final, coloca el pivote justo entre ambos grupos y devuelve su posición.



Quickselect: dos formas de elegir el pivote

Ambas reutilizan la misma partición de Lomuto; solo cambia cómo se decide el pivote.



Pivote ingenuo

Siempre toma el último elemento del segmento como pivote — igual que la partición de Lomuto clásica. Simple y directo.

Caso promedio

$O(n)$

Peor caso

$O(n^2)$



Mediana de medianas

Agrupar de 5 en 5, toma la mediana de cada grupo y recursivamente halla la mediana de esas medianas para usarla como pivote (Blum, Floyd, Pratt, Rivest & Tarjan, 1973).

Caso promedio

$O(n)$

Peor caso

$O(n)$



El corazón del algoritmo: lomuto_partition

```
src/lomuto.py

# Reordena arr[low..high] alrededor del pivote
def lomuto_partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i + 1
```

21

lomuto.py

líneas · 1 función · sin recursión

31

quickselect_naive.py

líneas · 1 función · iterativa

68

quickselect_mom.py

líneas · 3 funciones · recursión mutua



No todo es tiempo de ejecución: el costo de mantener cada versión

Archivo	Líneas	Funciones	Recursión
lomuto.py	21	1	No
quickselect_naive.py	31	1	No (iterativa)
quickselect_mom.py	68	3	Sí (mutua)

quickselect_mom.py tiene más del doble de líneas que la versión ingenua.



Búsqueda lineal del pivote

`_partition_around_value` debe ubicar el valor del pivote en el arreglo antes de particionar: un paso extra, fuente plausible de bugs.



Aritmética de grupos de 5

Bloques incompletos e índices ($\min(i+5, \text{high}+1)$) son un punto común de errores "off-by-one".



Lo que garantiza (y no garantiza) cada algoritmo

Partición de Lomuto

Caso promedio

$O(n)$

Peor caso

$O(n)$

Una sola pasada por el segmento.

Quickselect ingenuo

Caso promedio

$O(n)$

Peor caso

$O(n^2)$

Arreglo ordenado $\Rightarrow$ partición siempre desbalanceada.

Mediana de medianas

Caso promedio

$O(n)$

Peor caso

$O(n)$

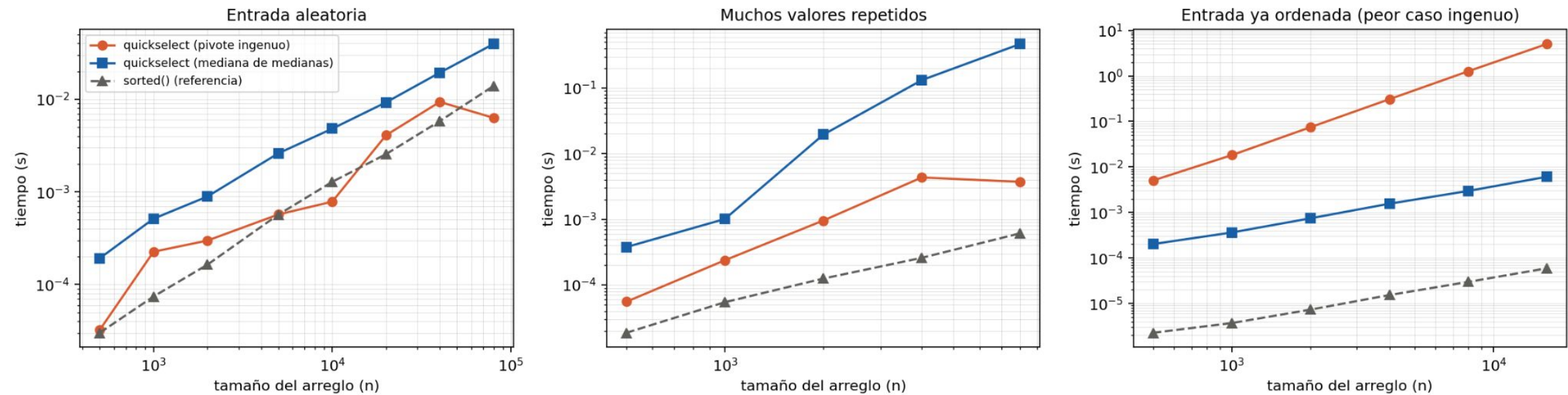
Cada partición deja $\leq 7n/10$ elementos del lado grande.

Fuentes: Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms*, cap. 9 (Medians and Order Statistics) · MIT OCW 6.046J, notas de clase 1 (Devadas & Leiserson) · MIT 18.310, *The Median Algorithm* (Shor)



La teoría, puesta a prueba con datos

Tiempo de ejecución: partición de Lomuto en quickselect, según estrategia de pivote



Como funciona la miniaplicación



1. Elegir estadística

Seleccionamos, por ejemplo, "aces por partido" entre las 5 estadísticas disponibles del top 50 ATP.



2. Mover el percentil

Con el slider (0–100) fijamos el percentil que queremos consultar, en vivo.



3. Ver el corte calculado

La app corre quickselect sobre Lomuto y marca visualmente quién queda dentro y quién fuera.



4. Probar un valor propio

Ingresamos un valor manual y la app responde en qué percentil quedaría ese jugador.



Repositorio y colaboración



`github.com/juanpablocarreram/ProyectoAlgoritmos`

Repositorio privado



Juan Pablo Carrera Martinez

juanpablo.carrera.m@gmail.com



Diego Sanchez Lopez

diegosan1019@gmail.com





¡Gracias!

Juan Pablo Carrera Martinez · Diego Sanchez Lopez

`github.com/juanpablocarreram/ProyectoAlgoritmos`